# 근사 알고리즘

APPROXIMATION ALGORITHMS

네 가지 공개 코딩 문제로 배우는 알고리즘

# 공개 학습 문제는 네 개입니다

6~11장, 챕터별 4문제씩 총 24문제 중 기말에 10문제 출제

| 번호 | 문제 | 난도 |
|---|---|---|
| 10-1 | 허용 배수 이내의 비용인지 확인 | 아주 쉬움 |
| 10-2 | 가장 한가한 기계에 작업 배정 | 쉬움 |
| 10-3 | 간선의 양 끝을 고르는 정점 커버 | 기본 |
| 10-4 | 새 원소를 가장 많이 덮는 집합 선택 | 조금 도전 |

기말은 별도 객관식·서술형 없이 코드 실행 결과로 평가합니다.

# 빨리 구한 해가 얼마나 좋은지 봅니다

최소화 문제에서는 최적 비용이 작을수록 좋습니다.

## 최적해

가능한 모든 해 중 비용이 가장 작은 해입니다.

## 근사해

최적해와의 차이를 일정한 배수 등으로 보장하는 해입니다.

## 이번 실습

주어진 절차를 구현하고 그 절차가 반환한 비용을 계산합니다.

# 정점 커버는 두 배 이내로 구성할 수 있습니다

아직 선택되지 않은 두 끝점을 한꺼번에 고릅니다.

## 한 번 선택

서로 끝점이 겹치지 않는 간선들을 하나씩 고르게 됩니다.

## 필요한 수

그 간선이 k개이면 어떤 정점 커버도 최소 k개가 필요합니다.

## 우리 절차

두 끝점씩 골라 2k개이므로 최적해의 두 배 이내입니다.

# 약 2시간 수업 흐름

한 문제를 이해하고 직접 구현하는 시간을 함께 확보합니다.

| 시간 | 내용 |
| --- | --- |
| 0–20분 | 허용 배수 확인 |
| 20–45분 | 기계 작업 배정 |
| 45–55분 | 쉬는 시간 |
| 55–80분 | 정점 커버 구성 |
| 80–105분 | 탐욕 집합 커버 |
| 105–120분 | 직접 구현·경계 테스트 |

마지막에는 빈 코드에서 다시 작성하고 테스트합니다.

# 10-1 허용 배수 이내의 비용인지 확인

아주 쉬움

최소화 문제에서 어떤 해의 비용 cost, 최적 비용 opt, 허용 배수 factor가 정수로 주어집니다. $1 \le opt \le cost \le 1,000,000$이고 $1 \le factor \le 100$입니다. cost가 opt의 factor배 이하이면 1, 넘으면 0을 반환하세요. 경계와 같아도 1입니다.

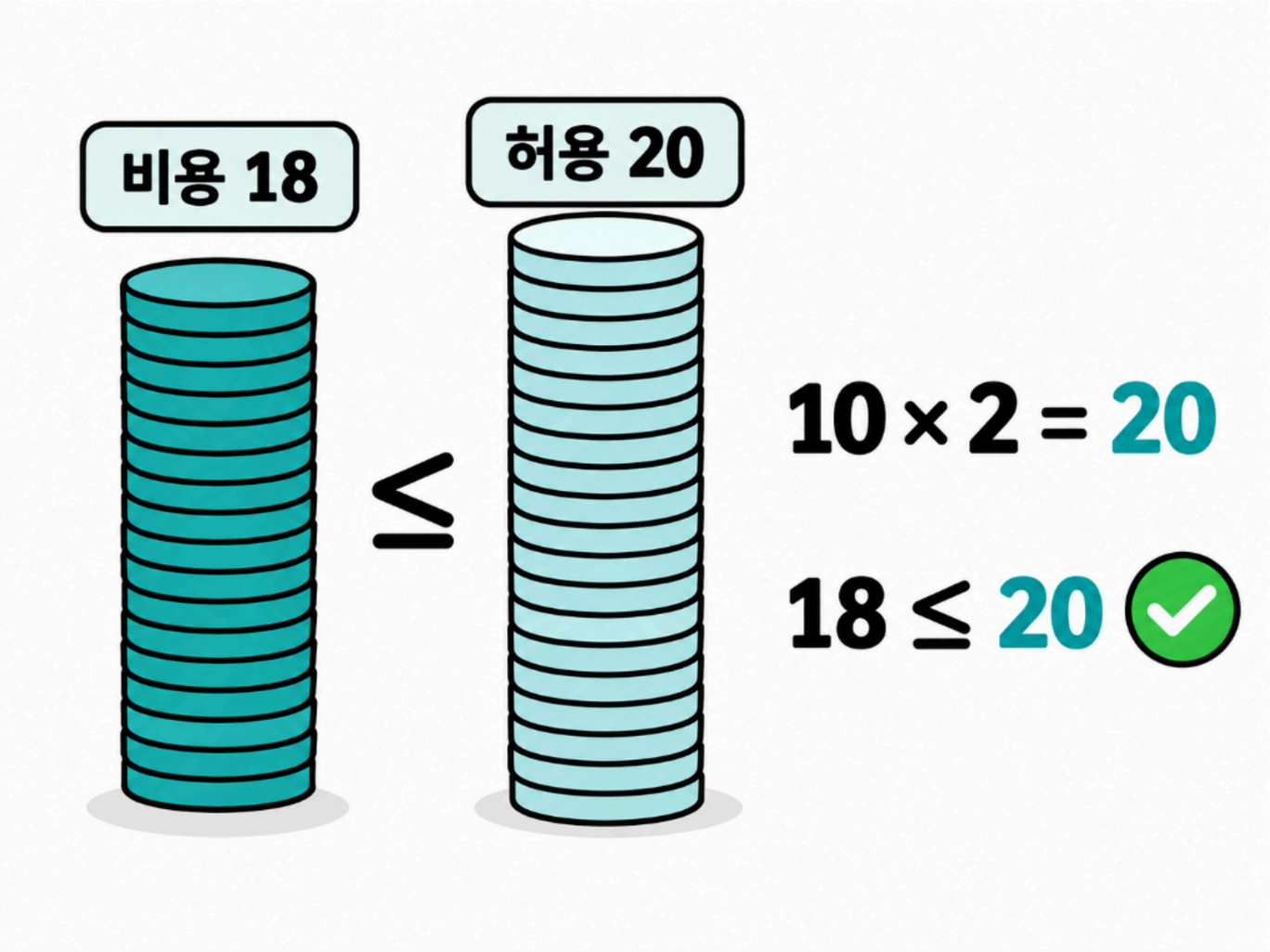
# 함수와 공개 예제

Python: within_factor(cost, opt, factor)

| 입력 | 반환값 |
|---|---|
| 18, 10, 2 | 1 |
| 21, 10, 2 | 0 |
| 20, 10, 2 | 1 |

int within_factor(int cost, int opt, int factor)

# 허용 배수 이내의 비용인지 확인의 작은 예




최적 비용 10
허용 배수 2
허용 비용은 20
18은 범위 안입니다.

# 작은 입력을 손으로 계산합니다

가능한 선택이나 중간 계산을 차례대로 확인합니다.

| cost | opt | factor | 허용 비용 | 반환 |
|---|---|---|---|---|
| 18 | 10 | 2 | 20 | 1 |
| 20 | 10 | 2 | 20 | 1 |
| 21 | 10 | 2 | 20 | 0 |

허용하는 가장 큰 비용 factor×opt를 계산하고 cost와 비교합니다. 나눗셈 없이 경계까지 정확하게 검사할 수 있습니다.

# 이 문제의 계산 방법

10-1

허용하는 가장 큰 비용 factor×opt를 계산하고 cost와 비교합니다. 나눗셈 없이 경계까지 정확하게 검사할 수 있습니다.

cost가 factor×opt와 같아도 허용합니다. 이 함수는 주어진 해 하나의 비용을 검사하며 최적값을 새로 찾지 않습니다.

# 허용 배수 이내의 비용인지 확인 · Python

```
def within_factor(cost, opt, factor):
    return int(cost <= factor*opt)
```

시간 O(1), 추가 공간 O(1)

# 허용 배수 이내의 비용인지 확인 · C

```
int within_factor(int cost, int opt, int factor) {
    return cost<=factor*opt;
}
```

시간 O(1), 추가 공간 O(1)

# 경계 입력과 복잡도

10-1

## 경계 입력

cost가 factor×opt와 같아도 허용합니다. 이 함수는 주어진 해 하나의 비용을 검사하며 최적값을 새로 찾지 않습니다.

## 계산량

시간 O(1), 추가 공간 O(1)

# 직접 작성하고 수정합니다

10-1 허용 배수 이내의 비용인지 확인

① 예상 반환값을 적습니다.
② 빈 코드에서 함수를 작성합니다.
③ 공개 예제와 경계 입력으로 실행합니다.
④ 틀린 입력을 고치고 다시 제출합니다.

# 10-2 가장 한가한 기계에 작업 배정

쉬움

기계 m대(1..10)가 모두 시간 0에 준비되어 있습니다. jobs의 작업을 주어진 순서대로, 현재 누적 작업 시간이 가장 작은 기계에 하나씩 배정하세요. 동률이면 번호가 작은 기계에 배정합니다. 작업 수는 0..1000, 각 시간은 0..1,000,000입니다. 작업은 나누지 않으며 기계는 받은 순서대로 처리합니다. 모든 작업이 끝나는 시각, 즉 기계별 누적 시간의 최댓값을 반환하세요. 작업이 없으면 0입니다.

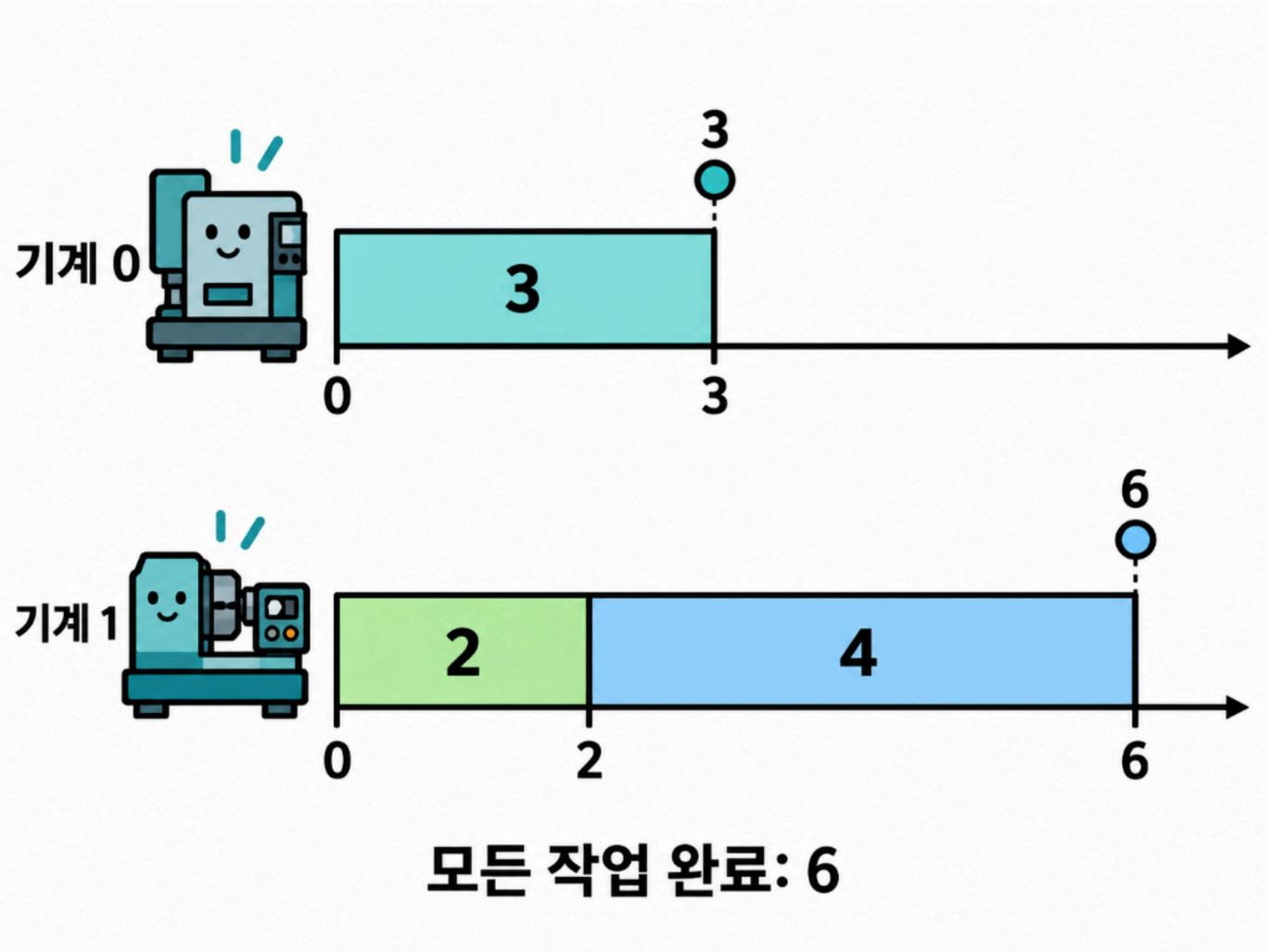
# 함수와 공개 예제

Python: list_scheduling(jobs, m)

| 입력 | 반환값 |
| --- | --- |
| [3,2,4], 2 | 6 |
| [], 3 | 0 |
| [2,2,2], 2 | 4 |

int list_scheduling(const int jobs[], int n, int m)

# 가장 한가한 기계에 작업 배정의 작은 예




작업 순서 3,2,4
기계0의 합은 3
기계1의 합은 6
모두 끝나는 때는 6

# 작은 입력을 손으로 계산합니다

가능한 선택이나 중간 계산을 차례대로 확인합니다.

| 배정할 작업 | 선택한 기계 | 기계0의 합 | 기계1의 합 |
|---|---|---|---|
| 3 | 0 | 3 | 0 |
| 2 | 1 | 3 | 2 |
| 4 | 1 | 3 | 6 |
| 완료 시각 | | | 6 |

기계마다 배정된 시간의 합을 저장합니다. 작업 하나마다 가장 작은 합을 찾아 그 작업의 시간을 더합니다.

# 이 문제의 계산 방법

10-2

기계마다 배정된 시간의 합을 저장합니다. 작업 하나마다 가장 작은 합을 찾아 그 작업의 시간을 더합니다.

입력 순서를 바꾸지 않습니다. 동률일 때 앞 번호를 유지하려면 더 작은 경우에만 기계를 바꿉니다. 평균이 아니라 가장 늦게 끝나는 시각을 반환합니다.

# 가장 한가한 기계에 작업 배정 · Python

```
def list_scheduling(jobs, m):
    load = [0]*m
    for time in jobs:
        machine = min(range(m), key=lambda i: load[i])
        load[machine] += time
    return max(load)
```

작업 n개, 기계 m대일 때 시간 O(nm), 공간 O(m)

# 가장 한가한 기계에 작업 배정 · C

```
int list_scheduling(const int jobs[], int n, int m) {
    int load[10]={0};
    for (int i=0; i<n; ++i) {
        int machine=0;
        for (int j=1; j<m; ++j)
            if (load[j]<load[machine]) machine=j;
        load[machine]+=jobs[i];
    }
    int answer=0;
    for (int j=0; j<m; ++j)
        if (load[j]>answer) answer=load[j];
    return answer;
}
```

작업 n개, 기계 m대일 때 시간 O(nm), 공간 O(m)

# 경계 입력과 복잡도

10-2

## 경계 입력

입력 순서를 바꾸지 않습니다. 동률일 때 앞 번호를 유지하려면 더 작은 경우에만 기계를 바꿉니다. 평균이 아니라 가장 늦게 끝나는 시각을 반환합니다.

## 계산량

작업 n개, 기계 m대일 때 시간 O(nm), 공간 O(m)

# 직접 작성하고 수정합니다

10-2 가장 한가한 기계에 작업 배정

① 예상 반환값을 적습니다.
② 빈 코드에서 함수를 작성합니다.
③ 공개 예제와 경계 입력으로 실행합니다.
④ 틀린 입력을 고치고 다시 제출합니다.

# 10분 쉬었다가 이어갑니다

앞의 두 계산을 정리합니다.

뒤의 문제는 앞에서 익힌 계산을
반복하거나 탐색에 연결합니다.

# 10-3 간선의 양 끝을 고르는 정점 커버

기본

정점 n개(1..100)의 무방향 그래프가 간선 배열 edges로 주어집니다. 각 간선은 서로 다른 정점 번호 두 개이며 번호는 0..n−1, 간선 수는 0..1000입니다. 간선을 입력 순서대로 보면서 양 끝이 모두 아직 선택되지 않았으면 두 정점을 모두 선택하세요. 한쪽이라도 이미 선택되어 있으면 그대로 넘어갑니다. 마지막에 선택한 정점 수를 반환하세요.

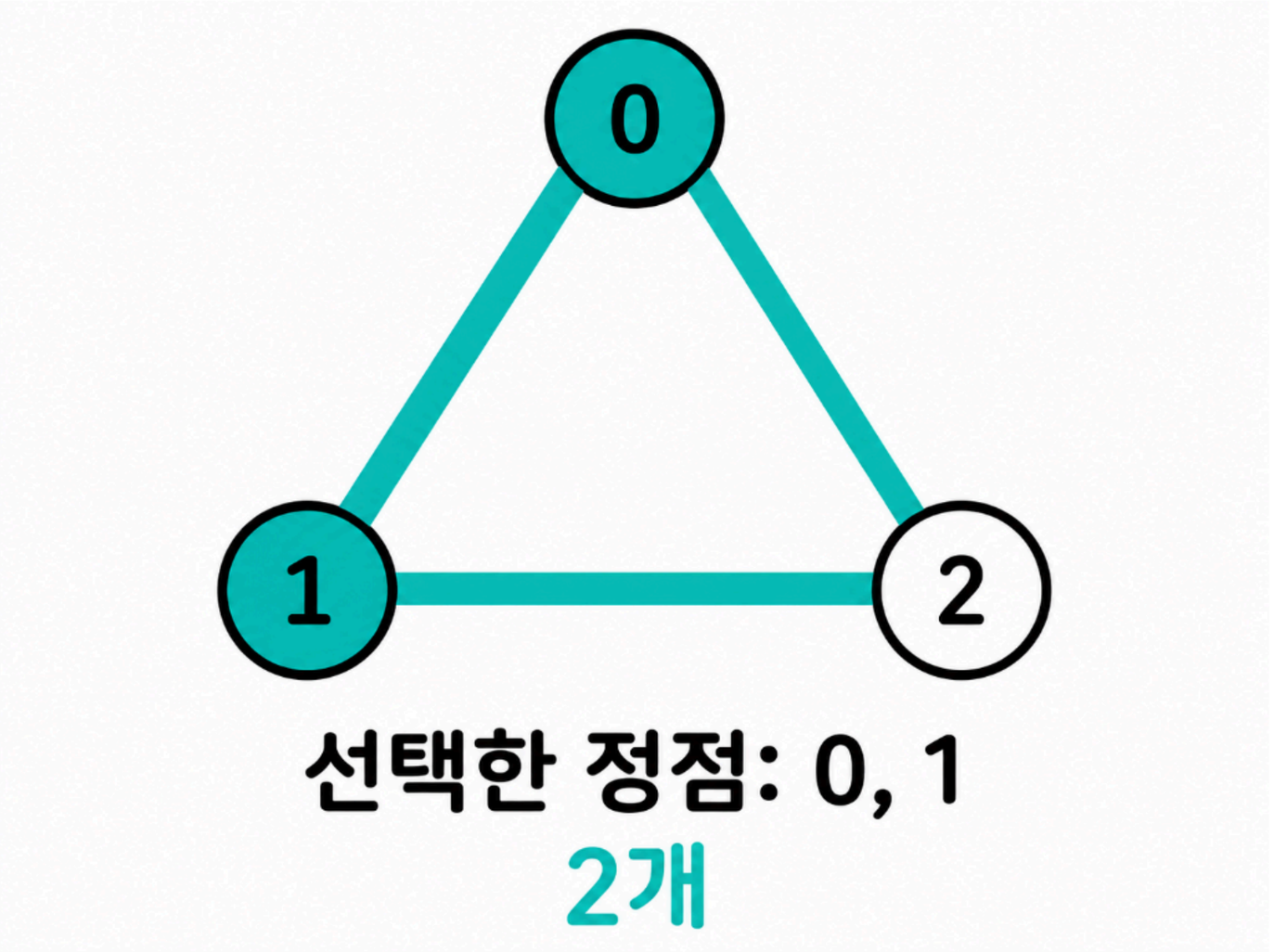
# 함수와 공개 예제

Python: cover_size(n, edges)

| 입력 | 반환값 |
|---|---|
| 3, [[0,1],[1,2],[0,2]] | 2 |
| 4, [[0,1],[2,3]] | 4 |
| 3, [] | 0 |

int cover_size(int n, const int edges[][2], int m)

# 간선의 양 끝을 고르는 정점 커버의 작은 예




첫 간선 0–1에서
양 끝을 고릅니다.
남은 간선도 덮여
선택한 수는 2입니다.

# 작은 입력을 손으로 계산합니다

가능한 선택이나 중간 계산을 차례대로 확인합니다.

| 간선 순서 | 처리 | 선택된 정점 |
| --- | --- | --- |
| 0–1 | 둘 다 미선택 → 둘 다 추가 | 0, 1 |
| 1–2 | 1이 선택됨 → 넘어감 | 0, 1 |
| 0–2 | 0이 선택됨 → 넘어감 | 0, 1 |
| 반환값 | 선택한 수 | 2 |

아직 덮이지 않은 간선을 만나면 두 끝점을 모두 고릅니다. 이미 한 끝점이라도 선택된 간선은 추가 선택 없이 넘어갑니다.

# 이 문제의 계산 방법

10-3

아직 덮이지 않은 간선을 만나면 두 끝점을 모두 고릅니다. 이미 한 끝점이라도 선택된 간선은 추가 선택 없이 넘어갑니다.

최소 개수 자체를 찾는 문제가 아니라 지정된 근사 절차를 구현합니다. 간선 입력 순서를 지키며 같은 정점을 중복 집계하지 않습니다.

# 간선의 양 끝을 고르는 정점 커버 · Python

```
def cover_size(n, edges):
    chosen = [False]*n
    for u, v in edges:
        if not chosen[u] and not chosen[v]:
            chosen[u] = chosen[v] = True
    return sum(chosen)
```

간선 m개일 때 시간 O(n+m), 공간 O(n)

# 간선의 양 끝을 고르는 정점 커버 · C

```
int cover_size(int n, const int edges[][2], int m) {
    int chosen[100]={0}, answer=0;
    for (int i=0; i<m; ++i) {
        int u=edges[i][0], v=edges[i][1];
        if (!chosen[u] && !chosen[v]) {
            chosen[u]=chosen[v]=1;
            answer+=2;
        }
    }
    return answer;
}
```

간선 m개일 때 시간 O(n+m), 공간 O(n)

# 경계 입력과 복잡도

10-3

## 경계 입력

최소 개수 자체를 찾는 문제가 아니라 지정된 근사 절차를 구현합니다. 간선 입력 순서를 지키며 같은 정점을 중복 집계하지 않습니다.

## 계산량

간선 m개일 때 시간 O(n+m), 공간 O(n)

# 직접 작성하고 수정합니다

10-3 간선의 양 끝을 고르는 정점 커버

① 예상 반환값을 적습니다.
② 빈 코드에서 함수를 작성합니다.
③ 공개 예제와 경계 입력으로 실행합니다.
④ 틀린 입력을 고치고 다시 제출합니다.

# 10-4 새 원소를 가장 많이 덮는 집합 선택

조금 도전

전체 원소는 0..n−1(n=1..20)입니다. 집합 0..30개가 정수 비트마스크 배열 sets로 주어집니다. 집합에 원소 k가 있으면 비트 k가 1이며 각 마스크는 $0..2^n-1$입니다. 아직 덮이지 않은 원소를 가장 많이 덮는 집합을 반복해서 고르세요. 동률이면 입력에서 먼저 나온 집합을 고릅니다. 전부 덮으면 선택 횟수, 더 덮을 수 없으면 −1을 반환하세요.

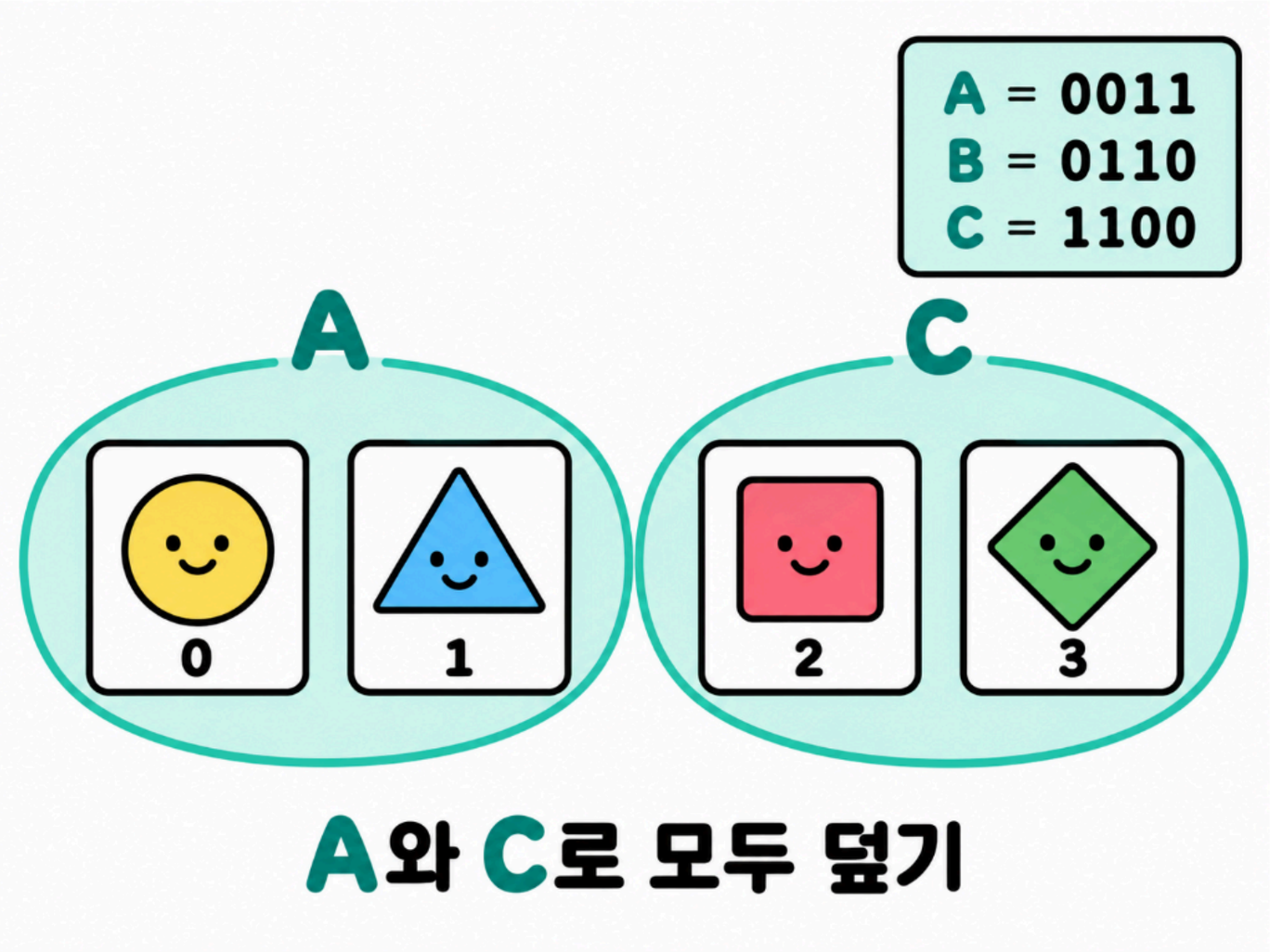
# 함수와 공개 예제

Python: greedy_set_cover(n, sets)

| 입력 | 반환값 |
| --- | --- |
| 4, [3,6,12] | 2 |
| 3, [1,2] | -1 |
| 3, [7] | 1 |

int greedy_set_cover(int n, const int sets[], int m)

# 새 원소를 가장 많이 덮는 집합 선택의 작은 예




처음 A로 0,1
다음 C로 2,3
네 원소를 모두 덮어
선택 횟수는 2입니다.

# 작은 입력을 손으로 계산합니다

가능한 선택이나 중간 계산을 차례대로 확인합니다.

| 남은 원소 | A={0,1} | B={1,2} | C={2,3} | 선택 |
|---|---|---|---|---|
| 0,1,2,3 | 2 | 2 | 2 | A (앞 순서) |
| 2,3 | 0 | 1 | 2 | C |
| 없음 | 0 | 0 | 0 | 2회로 종료 |

남은 원소를 비트마스크로 보관합니다. 각 집합과 AND한 결과의 1 개수가 새로 덮는 수이며,
그 수가 가장 큰 집합을 고릅니다.

# 이 문제의 계산 방법

10-4

남은 원소를 비트마스크로 보관합니다. 각 집합과 AND한 결과의 1 개수가
새로 덮는 수이며, 그 수가 가장 큰 집합을 고릅니다.

아무 집합도 새 원소를 덮지 못하면 무한 반복하지 않고 –1을 반환합니다.
매 단계 최소 한 원소가 줄어들어 반복은 최대 n회입니다.

# 집합도 0/1 비트로 표시할 수 있습니다

원소 0이 맨 오른쪽, 그 왼쪽이 원소 1, 2, 3입니다.

| 집합 | 포함한 원소 | 비트 표시 | 정수값 |
|---|---|---|---|
| A | 0, 1 | 0011 | 3 |
| B | 1, 2 | 0110 | 6 |
| C | 2, 3 | 1100 | 12 |
| 전체 | 0, 1, 2, 3 | 1111 | 15 |

비트 k가 1이면 그 집합에 원소 k가 들어 있습니다.

# 새로 덮는 수를 세고 남은 집합을 줄입니다

remaining은 아직 덮지 못한 원소들의 비트마스크입니다.

### 교집합: remaining & mask

두 비트가 모두 1인 자리만 남깁니다. 1100 & 0110은 0100입니다.

### 개수: value.bit_count()

정수의 1 비트 개수를 셉니다. 0100은 한 개입니다.

### 제거: remaining &= ~best

선택한 집합에 있는 원소를 remaining에서 지웁니다.

# 새 원소를 가장 많이 덮는 집합 선택 · Python

```
def greedy_set_cover(n, sets):
    remaining = (1 << n)-1
    count = 0
    while remaining:
        best, gain = 0, 0
        for mask in sets:
            added = (mask & remaining).bit_count()
            if added > gain:
                best, gain = mask, added
        if gain == 0: return -1
        remaining &= ~best
        count += 1
    return count
```

단순 비트 개수 계산 기준 시간 $O(mn^2)$, 공간 $O(1)$

# 새 원소를 가장 많이 덮는 집합 선택 · C

```
static int count_bits(int x) {
    int n=0; while (x) {n+=x&1; x>>=1;} return n;
}
int greedy_set_cover(int n, const int sets[], int m) {
    int remaining=(1<<n)-1, count=0;
    while (remaining) {
        int best=0, gain=0;
        for (int i=0; i<m; ++i) {
            int added=count_bits(sets[i]&remaining);
            if (added>gain) {best=sets[i]; gain=added;}
        }
        if (!gain) return -1;
        remaining&=~best; ++count;
    } return count;
}
```

단순 비트 개수 계산 기준 시간 $O(mn^2)$, 공간 $O(1)$

# 경계 입력과 복잡도

10-4

## 경계 입력

아무 집합도 새 원소를 덮지 못하면 무한 반복하지 않고 –1을 반환합니다. 매 단계 최소 한 원소가 줄어들어 반복은 최대 n회입니다.

## 계산량

단순 비트 개수 계산 기준 시간 $O(mn^2)$, 공간 $O(1)$

# 직접 작성하고 수정합니다

10-4 새 원소를 가장 많이 덮는 집합 선택

① 예상 반환값을 적습니다.
② 빈 코드에서 함수를 작성합니다.
③ 공개 예제와 경계 입력으로 실행합니다.
④ 틀린 입력을 고치고 다시 제출합니다.

# 이 장의 네 문제 정리

강의노트와 사이트는 같은 함수 규격을 사용합니다.

| 문제 | 함수 | 난도 |
|---|---|---|
| 10-1 | within_factor | 아주 쉬움 |
| 10-2 | list_scheduling | 쉬움 |
| 10-3 | cover_size | 기본 |
| 10-4 | greedy_set_cover | 조금 도전 |

공개 문제의 조건 안에서 올바른 반환값을 구합니다.

# 기말 준비

공개 24문제 중 10문제 출제

6~11장, 챕터마다 네 문제를 공부합니다.
정해진 함수 인자와 반환값을 지킵니다.
직접 작성한 코드의 실행 결과로 평가합니다.

# 근사 알고리즘 정리

APPROXIMATION ALGORITHMS

10-1 허용 배수 이내의 비용인지 확인
10-2 가장 한가한 기계에 작업 배정
10-3 간선의 양 끝을 고르는 정점 커버
10-4 새 원소를 가장 많이 덮는 집합 선택